

DOCUMENTACIÓN DEL PROYECTO

k0lmenIA

Agentes de QA para Claude Code: diseño de pruebas, automatización con k0lmena e integración con Xray, QMetry y AIO Tests — todo en español.

17 agentes

Web · API · Mobile · Performance

Xray · QMetry · AIO Tests

Jira · Confluence · Figma

Sin tokens para correr las suites

QARMY · Underc0de

github.com/underc0delabs/k0lmenIA

Licencia MIT

Contenido

- 1 Qué es k0lmenIA
- 2 Arquitectura
- 3 Cómo se usa: el flujo de trabajo
- 4 Agentes disponibles
- 5 Requisitos e instalación
- 6 Configuración: el archivo .env
- 7 Guía por tarea: diseño y ejecución manual
- 8 Automatización con k0lmena
- 9 Pruebas de performance
- 10 Gestión de pruebas: Xray, QMetry y AIO Tests
- 11 Conectores MCP: Jira, Confluence, Figma y más
- 12 Convenciones y estándares
- 13 Estructura del repositorio
- 14 Problemas frecuentes
- 15 Cómo extender el proyecto

1 Qué es k0lmenIA

k0lmenIA es un repositorio con un equipo de **agentes especializados en QA** para **Claude Code**. Está pensado para profesionales de testing manual: no hace falta saber programar, se trabaja conversando en lenguaje natural dentro de VS Code.

Vos ponés un insumo (una historia, una observación de bug, un contrato de API, o directamente una key de Jira o un link de Figma), le pedís a Claude Code lo que necesitás, y el agente que corresponde genera el resultado: análisis, planes, casos en Excel o Gherkin, datos, bugs, reportes HTML, automatización y carga en la herramienta de gestión.

Tres etapas, usás las que necesites

ETAPA	QUÉ RESUELVE	DÓNDE QUEDA
1. Diseño	Analizar historias, planificar, escribir casos manuales, BDD y de API, generar datos y redactar bugs.	output/
2. Ejecución y automatización	Ejecutar en vivo, o automatizar con k0lmena una sola vez y correr las suites con npm sin gastar tokens. Incluye pruebas de performance.	herramientas/k0lmena/
3. Gestión y cierre	Subir casos y ciclos a Xray, QMetry o AIO Tests, publicar resultados con evidencias y armar el informe de cierre.	Tu herramienta de gestión + output/

Principios

- **No inventar.** Si falta un paso, un dato, una regla de negocio o un umbral, el agente lo marca y lo pregunta. Es preferible un artefacto con huecos señalados que uno completo pero inventado.
- **Respetar los formatos.** Los campos, su orden y sus valores los definen las plantillas y los scripts, no el agente.
- **Trazabilidad.** Todo artefacto referencia la historia (HU-XXX) o el contrato del que sale, y cada caso automatizado lleva el ID del caso manual (@CP-001).
- **Cobertura pensada.** Positivos, negativos, bordes y validaciones de campos, no solo el camino feliz.
- **Ahorro de tokens.** Lo que se repite se automatiza una vez y después corre sin agentes.
- **Seguridad.** Los secretos viven en el .env de la raíz, que nunca se versiona; los agentes nunca piden tokens por el chat.

2 Arquitectura

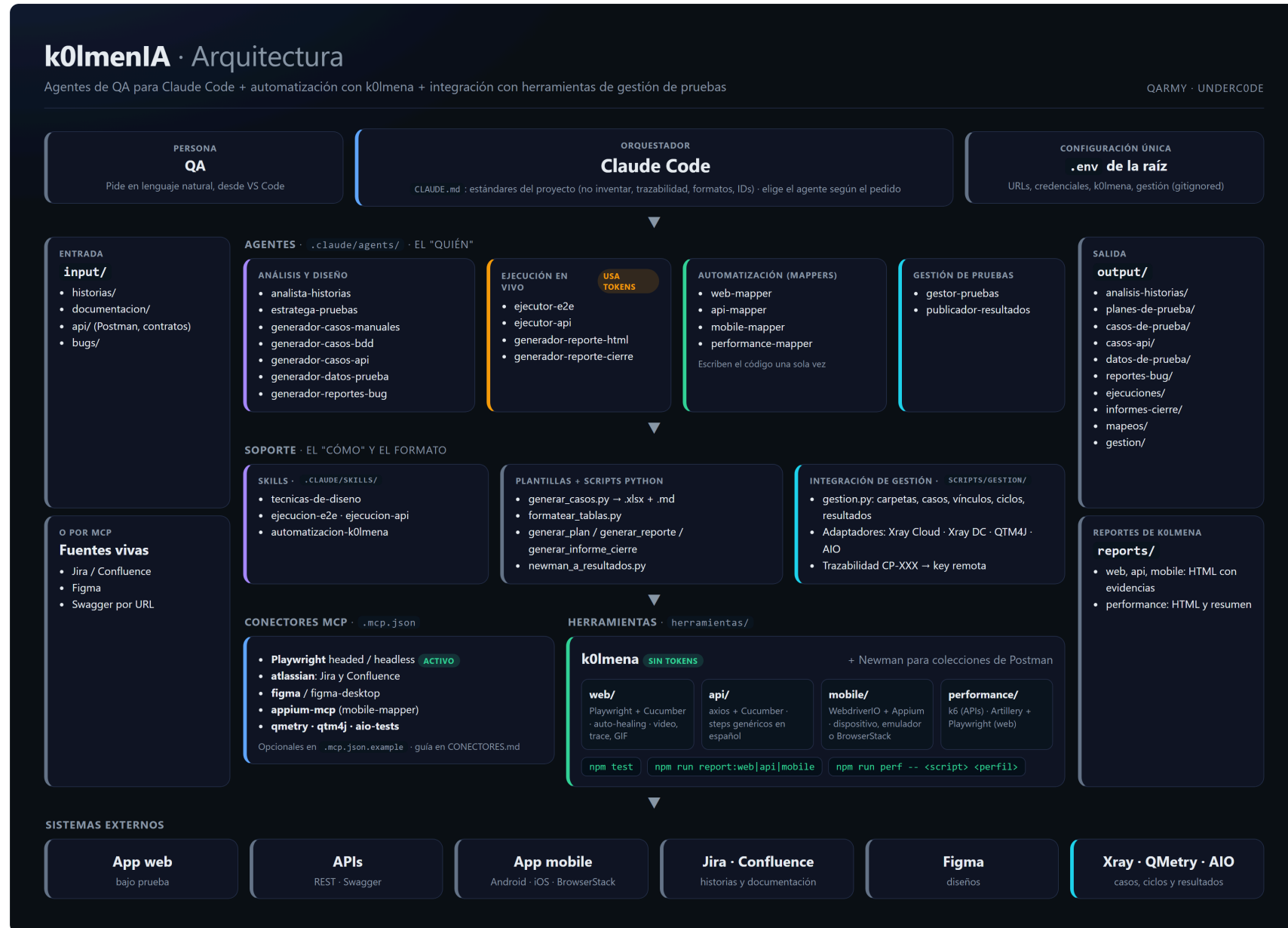


Figura 1 · Arquitectura de k0lmenIA

Las piezas

PIEZA	QUÉ ES	DÓNDE
Claude Code	Orquestador: interpreta el pedido, elige el agente y aplica los estándares de <code>CLAUDE.md</code> .	<code>CLAUDE.md</code>
Agentes	El "quién": un especialista por tarea (17 en total).	<code>.claude/agents/</code>
Skills	El "cómo": conocimiento que se carga cuando hace falta (técnicas de diseño, ejecución E2E y de API, convenciones de k0lmena).	<code>.claude/skills/</code>
Plantillas y scripts	Definen el formato de salida (planilla de casos, cobertura, bug) y generan los reportes HTML de forma determinística.	<code>plantillas/</code> , <code>scripts/</code>
Integración de gestión	<code>gestion.py</code> con un adaptador por herramienta: Xray Cloud, Xray Server/DC, QTM4J y AIO Tests.	<code>scripts/gestion/</code>
Conectores MCP	Conexiones a sistemas externos: Playwright (activo), Jira y Confluence, Figma, Appium, QMetry y AIO Tests.	<code>.mcp.json</code> , <code>.mcp.json.example</code>
Herramientas	Newman para colecciones de Postman y k0lmena para la automatización web, API, mobile y performance.	<code>herramientas/</code>
Entrada y salida	Insumos del usuario y artefactos generados, ordenados por tipo.	<code>input/</code> , <code>output/</code>

3 Cómo se usa: el flujo de trabajo

k0lmena · Cómo se usa

Le pedís a Claude Code lo que necesitás en lenguaje natural; el agente que corresponde hace el trabajo y deja el resultado

QARMY · UNDERCODE

1 Diseño

De la historia a los casos de prueba

0 Poné la historia en `input/historias/`
o pasá una key de Jira, una página de Confluence o un link de Figma
(con el conector activo)

1 "Analizó la historia HU-001"
analista-historias
→ ambigüedades y preguntas para el PO

2 "Armó el plan de pruebas de HU-001"
estratega-pruebas
→ plan HTML: alcance, riesgos, criterios

3 "Generó los casos de HU-001" · "Pasala a BDD"
generador-casos-manuales · generador-casos-bdd · generador-casos-api
→ casos-HU-001.xlsx + .md + cobertura · .feature

4 "Generó datos de prueba" · "Armó el reporte de este bug"
generador-datos-prueba · generador-reportes-bug
→ datos CSV/MD · BUG-001.md

UNA SOLA CONFIGURACIÓN

El `.env` de la raíz: URLs, credenciales, k0lmena y gestión.
Nunca se commitea.

NO INVENTA

Si falta un paso, un dato o un umbral, el agente lo marca y te lo pregunta.

AHORRA TOKENS

Lo que se repite se automatiza una vez y corre con `npm`, sin agentes.

TRAZABILIDAD

HU-001 → CP-001 → @CP-001 → key en Xray/QMetry/AIO
→ resultado.

2 Ejecución y automatización

Una vez con agentes, después sin tokens

5 "Automatizó en k0lmena los casos de HU-001 contra `https://tu-app.com`"
web-mapper · api-mapper (Postman / Swagger) · mobile-mapper (Appium)
→ recorre la app, ajusta los pasos y escribe `.feature` + `steps` + `locators` **UNA VEZ**

6 Corrés la suite cuando quieras **SIN TOKENS**

```
cd herramientas/k0lmena
npm test # web + api (TAGS=@HU-001)
npm run test:mobile
npm run report:web # HTML con evidencias
```

7 "Armó una prueba de carga del login para 50 usuarios"
performance-mapper: pregunta ambiente, carga y umbrales · k6 o Artillery

```
npm run perf -- HU-001-login smoke
npm run perf -- HU-001-login load # pide confirmación
```

¿Una prueba puntual que no vale la pena automatizar? → "Ejecutó el escenario X contra `https://...`" (ejecutor-e2e / ejecutor-api, en vivo)

3 Gestión y cierre

Xray · QMetry (QTM4J) · AIO Tests

8 "Subí los casos de HU-001 a Xray en 'HU-001 Registro',
vinculados a PROJ-12"
gestor-pruebas
→ carpeta + casos vinculados a la historia

9 "Creó el ciclo Sprint 5 con esos casos"
gestor-pruebas
→ ciclo (Test Execution) vinculado a PROJ-12

10 "Subí los resultados de la última corrida al ciclo PROJ-60"
publicador-resultados
→ estado + error + captura, GIF y video (si falló) por caso

11 "Armó el informe de cierre de HU-001"
generador-reporte-cierre
→ resultados, bugs, riesgos y go / no-go

Figura 2 · Flujo de uso de punta a punta

Paso a paso

1. Poné tus insumos en `input/` (o pasá una key de Jira, una página de Confluence o un link de Figma con el conector activo).
2. Pedile a Claude Code lo que necesitás, en lenguaje natural. Elige solo el agente adecuado; también podés nombrarlo ("usá el web-mapper para...").
3. Revisá el resultado en `output/` o en `herramientas/k0lmena/`.
4. Lo que se repite, automatizalo y corrélo con `npm test`, sin tokens.

PASO	LE PEDÍS	OBTENÉS
1	"Analizá la historia HU-001"	Ambigüedades y preguntas para el PO
2	"Armá el plan de pruebas de HU-001"	Plan HTML
3	"Generá los casos de HU-001"	<code>casos-HU-001.xlsx</code> + cobertura
4	"Subí los casos a Xray en 'HU-001 Registro', vinculados a PROJ-12, y creá el ciclo Sprint 5"	Casos y ciclo en Xray
5	"Automatizá en k0lmena los casos de HU-001 contra https://tu-app.com"	<code>.feature</code> + steps + locators
6	<code>npm test</code> y <code>npm run report:web</code>	Reporte HTML con evidencias
7	"Subí los resultados al ciclo del Sprint 5"	Estados y evidencias en Xray
8	"Armá una prueba de carga del login para 20 usuarios"	Script k6/Artillery + reporte HTML
9	"Armá el informe de cierre de HU-001"	Go / no-go

4 Agentes disponibles

Análisis y diseño

AGENTE	QUÉ HACE	SALIDA
analista-historias	Analiza historias y criterios de aceptación: ambigüedades, vacíos, riesgos y preguntas de refinamiento.	output/analisis-historias/
estratega-pruebas	Plan de pruebas (alcance, riesgos, tipos de prueba, entorno, criterios) como dashboard HTML.	output/planes-de-prueba/
generador-casos-manuales	Casos en Excel y Markdown + informe de cobertura con preguntas para el PO.	output/casos-de-prueba/manuales/
generador-casos-bdd	Escenarios Gherkin (keywords en inglés, contenido en español) + cobertura por criterio.	output/casos-de-prueba/bdd/
generador-casos-api	Casos de API (positivos, negativos, schema, autorización) desde un contrato.	output/casos-api/
generador-datos-prueba	Datos válidos, inválidos y de borde (Markdown o CSV).	output/datos-de-prueba/
generador-reportes-bug	Reporte de bug profesional según la plantilla, con severidad y prioridad justificadas.	output/reportes-bug/

Ejecución en vivo

USA TOKENS EN CADA CORRIDA

AGENTE	QUÉ HACE	SALIDA
ejecutor-e2e	Ejecuta casos o escenarios en un navegador real con Playwright MCP (headed o headless), con evidencia.	output/ejecuciones/
ejecutor-api	Corre una colección de Postman con Newman.	output/ejecuciones/
generador-reporte-html	Dashboard HTML de los resultados de una ejecución.	output/ejecuciones/
generador-reporte-cierre	Informe de cierre de la ronda: resultados, bugs, riesgos y go/no-go.	output/informes-cierre/

Automatización con k0lmena

DESPUÉS CORRE SIN TOKENS

AGENTE	QUÉ HACE	SALIDA
web-mapper	Recorre la web siguiendo los casos y genera <code>.feature</code> , steps y locators.	k0lmena/web/
api-mapper	Lee Postman o Swagger/OpenAPI, verifica los endpoints y genera los <code>.feature</code> de API.	k0lmena/api/

<code>mobile-mapper</code>	Recorre la app con Appium siguiendo los casos y genera <code>.feature</code> , steps y locators.	<code>k0lmena/mobile/</code>
<code>performance-mapper</code>	Pruebas de carga, estrés, soak y picos con k6 o Artillery; pregunta carga y umbrales.	<code>k0lmena/performance/</code>

Gestión de pruebas

AGENTE	QUÉ HACE	SALIDA
<code>gestor-pruebas</code>	Crea carpetas y casos en Xray, QMetry (QTM4J) o AIO Tests, los vincula a la historia, crea ciclos y les agrega casos.	La herramienta + <code>output/gestion/</code>
<code>publicador-resultados</code>	Sube los resultados de <code>npm test</code> al ciclo: estado, error y evidencias.	La herramienta

5 Requisitos e instalación

Requisitos según lo que vayas a usar

PARA...	NECESITÁS
Usar los agentes	Claude Code y una cuenta de Claude (Pro, Max, Team o Enterprise) o acceso por API de Anthropic. VS Code recomendado.
Generar casos, reportes y usar la gestión	Python 3 con <code>openpyxl</code> , <code>tabulate</code> y <code>requests</code> (<code>pip install -r requirements.txt</code>).
Ejecutar E2E en vivo	Node.js 18+ y los navegadores de Playwright.
Ejecutar API con Newman	Newman (<code>npm install -g newman</code>).
Automatizar con k0lmena	Node.js 20+.
Automatizar mobile	Node.js 22+, JDK y Android SDK, o macOS con Xcode para iOS; o una cuenta de BrowserStack.

Instalación paso a paso

1. Instalar Claude Code

```
curl -fsSL https://claude.ai/install.sh | bash      # macOS / Linux
npm install -g @anthropic-ai/claude-code          # alternativa con Node.js 18+
claude --version
```

En Windows, seguí la guía oficial en docs.claude.com/en/docs/claude-code/overview.

2. Clonar el repositorio

```
git clone https://github.com/underc0delabs/k0lmenIA.git
cd k0lmenIA
pip install -r requirements.txt
cp .env.example .env      # y completalo (sección 6)
```

3. Abrir en VS Code y lanzar Claude Code

```
code .
claude      # en la terminal integrada; la primera vez pide autenticarte
```

4. Instalar lo opcional

```
npx playwright install           # ejecución E2E en vivo
npm install -g newman            # ejecución de colecciones de Postman
cd herramientas/k0lmena
npm install                      # k0lmena
npx playwright install chromium
npm run bootstrap:k6             # solo para performance de APIs con k6
```

6 Configuración: el archivo .env

Hay **un solo** `.env`, **en la raíz del repo**. Lo usan los agentes, k0lmena y los scripts de gestión. Está en `.gitignore` y nunca se sube. La plantilla comentada es `.env.example`.

GRUPO	VARIABLES	PARA QUÉ
App bajo prueba	<code>APP_URL</code> , <code>APP_USER</code> , <code>APP_PASSWORD</code>	Ejecución E2E y logins de la automatización
API	<code>API_TOKEN</code> , <code>API_BASEURL</code>	Newman, suite de API y k6
k0lmena web	<code>BASEURL</code> (vacía = <code>APP_URL</code>), <code>BROWSER</code> , <code>HEADLESS</code> , <code>VIEWPORT_WIDTH</code> , <code>VIEWPORT_HEIGHT</code> , <code>LOCALE</code> , <code>TIMEZONE</code>	Navegador de la suite web
Ejecución	<code>TAGS</code> , <code>PARALLEL</code>	Qué escenarios y cómo se corren
Evidencias	<code>EVIDENCE</code> (<code>captura</code> , <code>ambos</code> , <code>off</code> ; vacía = se pregunta), <code>VIDEO</code> , <code>TRACE</code>	Qué se adjunta al reporte
Auto-healing	<code>K0LMENA_AUTO_HEALING</code> (<code>off</code> , <code>learn</code> , <code>heal</code> , <code>on</code>)	Recuperar locators rotos
Mobile	<code>MOBILE_TARGET</code> , <code>MOBILE_PLATFORM</code> , <code>MOBILE_DEVICE_NAME</code> , <code>MOBILE_APP</code> , <code>MOBILE_UDID</code> , <code>BROWSERSTACK_*</code>	Dónde corre la suite mobile
Performance	<code>PERF_VUS</code> , <code>PERF_DURACION</code> (opcionales)	Pisar la carga de los scripts k6
Gestión	<code>GESTION_HERRAMIENTA</code> , <code>GESTION_PROYECTO</code> + credenciales (<code>XRAY_CLOUD_*</code> y <code>JIRA_*</code> , <code>XRAY_DC_*</code> , <code>QTM4J_*</code> o <code>AIO_API_TOKEN</code>)	Xray, QMetry y AIO Tests

Usá credenciales de un entorno de prueba, nunca de producción. Los agentes leen los secretos del `.env`: nunca pegues un token en el chat ni lo escribas en un archivo versionado.

Carpetas de entrada

CARPETA	QUÉ VA
<code>input/historias/</code>	Historias de usuario con sus criterios de aceptación (<code>HU-001.md</code> , ...).
<code>input/documentacion/</code>	Documentación del producto que sirva de contexto.
<code>input/api/</code>	Contratos de API, colecciones de Postman y environments (incluye una de ejemplo).
<code>input/bugs/</code>	Observaciones sueltas de errores para convertir en reportes.

7 Guía por tarea: diseño y ejecución manual

Analizar una historia · `analista-historias`

"Analizó la historia HU-001 y decime qué ambigüedades tiene."

Revisa cada criterio de aceptación y entrega `analisis-HU-001.md` con ambigüedades, vacíos, riesgos, casos no contemplados y preguntas de refinamiento para el PO.

Armar el plan de pruebas · `estratega-pruebas`

"Armá el plan de pruebas de HU-001."

Dashboard HTML en modo oscuro con alcance, objetivos, tipos de prueba, riesgos, datos y entorno, y criterios de entrada y salida.

Generar casos manuales · `generador-casos-manuales`

"Generá los casos de prueba de HU-001."

Entrega `casos-HU-001.xlsx` (la planilla, con el formato de `plantillas/plantilla-casos-prueba.xlsx`), `casos-HU-001.md` y `casos-HU-001-cobertura.md`. Cubre camino feliz, negativos, bordes y validaciones. Esta planilla es la que después se sube a la herramienta de gestión o se automatiza.

Generar escenarios BDD · `generador-casos-bdd`

"Pasá la historia HU-001 a escenarios BDD."

Genera `HU-001-<slug>.feature` (un tag `@CP-XXX` por escenario) y `HU-001-cobertura.md`.

Generar casos de API · `generador-casos-api`

"Generá los casos de la API de autenticación a partir de `input/api/auth-endpoints.md`."

Tabla resumen y detalle de cada caso: método, endpoint, headers, body, status y schema esperado.

Datos de prueba y reportes de bug

"Generá datos de prueba para el formulario de registro, en CSV." · "Tomá la observación de input/bugs/ y armá el reporte de bug."

Los datos incluyen válidos, inválidos y de borde. El bug sigue `plantillas/plantilla-reporte-bug.md` (editala para usar tu formato) y marca lo que falta en vez de inventarlo.

Ejecutar en vivo · `ejecutor-e2e` y `ejecutor-api`

"Ejecutá SOLO el escenario de registro válido de HU-001 contra https://tu-app.com." · "Ejecutá la colección de input/api/ contra https://tu-api.com."

El ejecutor E2E pregunta si querés ver el navegador, ejecuta con Playwright MCP, saca evidencia y genera `reporte-HU-001-<fecha-hora>.html`. El de API corre la colección con Newman. Ideal para pruebas puntuales; lo que se repite conviene automatizarlo.

Reportes de ejecución e informe de cierre

"Armá el reporte HTML de la última ejecución." · "Armá el informe de cierre de las pruebas de HU-001."

Dashboards con indicadores, gráficos y detalle; el informe de cierre agrega bugs abiertos, riesgos y la recomendación go/no-go.

8 Automatización con k0lmena

k0lmena es el framework de automatización integrado en `herramientas/k0lmena/`: **web** (Playwright + Cucumber), **API** (axios + Cucumber), **mobile** (WebdriverIO + Appium) y **performance** (k6 y Artillery). Un agente *mapper* recorre la aplicación una sola vez para verificar los casos y escribe la automatización; desde ahí, la suite corre con `npm` sin agentes y sin tokens. Las carpetas vienen vacías: todo lo generan los mappers para tu aplicación.

web-mapper

"Automatizá en k0lmena los casos de `output/casos-de-prueba/manuales/casos-HU-001.xlsx` contra `https://tu-app.com`."

- 1. Lee los casos (manuales o BDD).
- 2. Navega la web real con Playwright MCP y comprueba que cada paso se puede ejecutar.
- 3. Genera `web/features/HU-001-<slug>.feature`, `web/steps/HU-001.steps.ts` y `web/locators/HU-001.locators.ts`, reutilizando steps existentes.
- 4. Valida con `npm run test:web` y deja `output/mapeos/mapeo-HU-001-web.md` (pasos mapeados, ajustados y bloqueados).

Si un paso no se puede ejecutar, el escenario queda `@bloqueado` con el motivo y no corre hasta que se resuelva.

api-mapper

"Pasá a k0lmena los endpoints de `/pet` del Swagger `https://petstore.swagger.io/v2/swagger.json`."

Lee Postman (v2.x) o Swagger/OpenAPI, muestra los endpoints para elegir, diseña escenarios positivos, negativos y de autorización, los verifica contra la API real y escribe los `.feature` con los **steps genéricos en español** del framework. Antes de un POST, PUT, PATCH o DELETE pide autorización.

mobile-mapper

"Automatizá en k0lmena los casos de login de la app en el emulador."

Necesita el conector Appium MCP y la app en `mobile/apps/` (no se versiona). Después la suite corre sin MCP en:

MOBILE_TARGET	QUÉ NECESITÁS
<code>device</code>	Dispositivo por USB con depuración y <code>MOBILE_UDID</code> (sale de <code>adb devices</code>).
<code>emulator</code>	Emulador o simulador encendido y <code>MOBILE_DEVICE_NAME</code> .
<code>browserstack</code>	<code>BROWSERSTACK_USER</code> , <code>BROWSERSTACK_KEY</code> y <code>BROWSERSTACK_APP</code> (<code>bs://...</code>).

Correr la suite (sin tokens)

```
cd herramientas/k0lmena
npm test # web + API
npm run test:web | test:api | test:mobile | test:all
TAGS=@HU-001 npm test # bash: filtrar por tag
$env:TAGS="@HU-001"; npm test # PowerShell
npm run report:web | report:api | report:mobile # reporte HTML de la última corrida
```

Si `EVIDENCE` está vacía, `npm test` pregunta al arrancar qué guardar de los tests que pasan: solo captura, o captura + GIF del recorrido.

Evidencias en el reporte

SUITE	ESCENARIO QUE PASA	ESCENARIO QUE FALLA
Web	Captura final (+ GIF opcional)	Video completo, captura, error con stack, URL y título, logs del navegador (consola, errores JS, requests fallidos, HTTP $\geq$ 400), logs de Node y trace de Playwright
API	Request y response (el <code>Authorization</code> se oculta)	Lo mismo, más la validación que falló
Mobile	Captura final (+ GIF opcional)	Video completo, captura, page source y logs del dispositivo

Herramientas extra

COMANDO	PARA QUÉ
<code>npm run crawler [url]</code>	Genera un archivo de locators recorriendo una página.
<code>npm run link-tester</code>	Busca enlaces e imágenes rotas en <code>BASEURL</code> .
<code>npm run record</code>	Graba un flujo con Playwright codegen.
<code>npm run debug</code>	Corre la suite web en modo debug.

Con `K0LMENA_AUTO_HEALING=on`, si un locator deja de funcionar, k0lmena prueba alternativas aprendidas y lo informa como `HEALED: <original> -> <candidato>`.

9 Pruebas de performance

performance-mapper

"Armá una prueba de carga de los endpoints de /pet para 50 usuarios." · "Quiero medir el login con 10 usuarios simultáneos en el navegador."

1. **Pregunta** (sin valores por defecto): ambiente y confirmación de que no es producción, flujo o endpoints, carga objetivo, **umbrales** (p95, p99, % de error), perfiles y si puede usar métodos con efecto.
2. **Elige la herramienta**: k6 para APIs (escala a miles de usuarios) o Artillery + Playwright para flujos de navegador (pocos usuarios: cada uno es un Chromium).
3. **Escribe el script** una vez, lo valida con un smoke y, si se lo pedís, corre la prueba con carga **previa confirmación** de cada corrida.
4. Deja `output/mapeos/mapeo-HU-XXX-performance.md`.

Comandos

```
npm run perf                # lista los scripts
npm run perf -- <script> smoke    # valida el script con carga mínima
npm run perf -- <script> load      # carga objetivo (pide confirmación)
npm run perf -- <script> stress --confirmar  # sin pregunta: CI o autorización previa
npm run perf -- <script> load --vus 20 --duracion 2m  # k6: pisa la carga
```

PERFIL	QUÉ HACE
smoke	Carga mínima para validar que el script funciona.
load	Sube hasta la carga objetivo, la sostiene y baja.
stress	1x, 2x y 3x la carga objetivo: busca el punto de quiebre.
soak	Carga objetivo durante mucho tiempo: fugas de memoria y degradación.
spike	Salto brusco a un pico y vuelta: cómo se recupera.

Cada corrida deja en `reports/performance/<k6|artillery>/` un reporte HTML (veredicto, requests, req/s, errores, latencias p50/p95/p99, umbrales con el valor medido, detalle por endpoint o paso, códigos de respuesta, checks, errores y, en Artillery, la evolución en el tiempo), un resumen JSON, el resultado crudo y el log. Si un umbral no se cumple, el comando termina con error.

Una prueba de carga genera tráfico real. Corrédla solo contra ambientes de prueba o con autorización de los dueños del sistema.

10 Gestión de pruebas: Xray, QMetry y AIO Tests

Los agentes `gestor-pruebas` y `publicador-resultados` trabajan con **Xray Cloud**, **Xray Server/Data Center**, **QMetry para Jira (QTM4J) Cloud** y **AIO Tests**, con los mismos comandos para todas. Se configura en el `.env`: `GESTION_HERRAMIENTA` (`xray-cloud`, `xray-dc`, `qtm4j` o `aio`), `GESTION_PROYECTO` (key de Jira) y las credenciales de la herramienta. En los comandos, las carpetas van sin barra inicial (en Git Bash, `/HU-001` se convierte en una ruta de Windows); referencia completa en `scripts/gestion/README.md`.

gestor-pruebas

"Subí los casos de HU-001 a Xray en la carpeta 'HU-001 Registro', vinculados a PROJ-12." · "Creá el ciclo 'Sprint 5 - Regresión' con CP-001 a CP-005 y vincúlalo a PROJ-12."

Crea la carpeta, sube los casos (manuales desde el `.xlsx` o BDD desde el `.feature`), los vincula a la historia, crea ciclos y les agrega casos. La primera vez hace una prueba en seco (`--dry-run`). Guarda la trazabilidad en `output/gestion/HU-001-<herramienta>.json`: un caso nunca se sube dos veces.

publicador-resultados

"Subí los resultados de la última corrida web al ciclo PROJ-60."

Lee el reporte de k0lmena, asocia cada escenario a su caso por el tag `@CP-XXX` y carga el estado, un comentario con el error y las evidencias: captura siempre, GIF si se generó y video solo si falló. Nunca asigna un resultado por aproximación.

Equivalencias entre herramientas

CONCEPTO	XRAY	QTM4J	AIO TESTS
Caso	Issue Test (Manual o Cucumber)	Test case	Caso (Classic o BDD)
Ciclo	Test Execution	Test cycle	Test cycle
Caso ↔ historia	Link "Test" (cobertura)	Requirement link	jiraRequirementIDs
Ciclo ↔ historia	Link "Relates"	Requirement link	jiraTaskIDs

Comandos manuales (sin agentes)

```
python scripts/gestion/gestion.py --dry-run carpeta --ruta "HU-001 Registro"
python scripts/gestion/gestion.py subir-casos --origen <casos.xlsx> --carpeta "HU-001 Registro" --
historia PROJ-12 --traza HU-001
python scripts/gestion/gestion.py crear-ciclo --nombre "Sprint 5" --historia PROJ-12 --traza HU-001
python scripts/gestion/gestion.py extraer-resultados --reporte <cucumber-report.json>
python scripts/gestion/gestion.py publicar-resultados --ciclo PROJ-60 --resultados <resultados.json>
--traza HU-001
```

11 Conectores MCP: Jira, Confluence, Figma y más

Los servers MCP conectan a Claude Code con sistemas externos. `.mcp.json` trae activo **Playwright** (headed y headless). Los demás están en `.mcp.json.example` :

CONECTOR	PARA QUÉ	AUTENTICACIÓN
<code>atlassian</code>	Jira y Confluence: leer historias, criterios y documentación funcional directamente.	Cuenta (OAuth)
<code>figma</code> / <code>figma-desktop</code>	Leer diseños desde un link a un frame o archivo: textos, estados y componentes para diseñar casos o comparar la app con el diseño. Requiere asiento Full o Dev.	Cuenta (OAuth)
<code>appium-mcp</code>	Lo usa el <code>mobile-mapper</code> para recorrer la app.	<code>ANDROID_HOME</code>
<code>qmetry</code> / <code>qtm4j</code>	Consultar QMetry desde el chat.	API key
<code>aio-tests</code>	Consultar AIO Tests desde el chat.	Token

Activar un conector

1. Copiá su entrada de `.mcp.json.example` a `.mcp.json`.
2. Agregá su nombre a `enabledMcpjsonServers` en `.claude/settings.local.json`.
3. Si usa token, definí la variable en el entorno antes de lanzar `claude`. Atlassian y Figma se autorizan con tu cuenta desde `/mcp`.
4. Reiniciá Claude Code y verificá con `/mcp`.

Con `atlassian` o `figma` activos, cualquier agente acepta como insumo una key de Jira, una página de Confluence o un link de Figma:

"Generá los casos de PROJ-12 usando la página de Confluence 'Reglas de facturación' y este diseño de Figma."

Para Xray no hace falta MCP: se usa `scripts/gestion/`. Guía completa en `CONECTORES.md`.

12 Convenciones y estándares

ELEMENTO	FORMATO	ESCALA	VALORES
Historias	HU-001	Severidad y prioridad	Crítica · Alta · Media · Baja
Criterios de aceptación	CA1 , CA2		
Casos manuales	CP-001		
Casos de API	CP-API-001	Estado de un caso	N/A · Pendiente · En ejecución · Aprobado · Fallido · Bloqueado
Bugs	BUG-001		

Tags en k0lmena

- **Feature:** la historia y el tipo (@HU-001 @web , @api , @mobile).
- **Scenario:** el ID del caso de origen (@CP-001 , @CP-API-001) y @Smoke si es crítico.
- @bloqueado : un paso no se pudo verificar; el escenario no se ejecuta.

Nombres de archivos

ARTEFACTO	NOMBRE
Análisis	analisis-HU-001.md
Casos manuales	casos-HU-001.xlsx + .md + -cobertura.md
Casos BDD	HU-001-registro.feature + HU-001-cobertura.md
Bug	BUG-001.md
Automatización	<tipo>/features/HU-001-<slug>.feature , steps/HU-001.steps.ts , locators/HU-001.locators.ts
Performance	performance/k6/http/HU-001-<slug>.ts o performance/artillery/HU-001-<slug>.yaml
Reporte de mapeo	output/mapeos/mapeo-HU-001-<web api mobile performance>.md

Las tablas de los .md que generan los agentes se normalizan con python scripts/formatear_tablas.py <archivo> , sin emojis en las celdas.

13 Estructura del repositorio

```
k0lmenIA/
├─ CLAUDE.md           Contexto y estándares (Claude Code lo lee siempre)
├─ ARQUITECTURA.md    Cómo crece el repo
├─ CONECTORES.md       Cómo activar los conectores MCP
├─ .env.example         Plantilla de variables (copiar a .env)
├─ .mcp.json            Conexiones MCP activas (Playwright)
├─ .mcp.json.example    Conectores opcionales
├─ requirements.txt     Dependencias de Python
├─ docs/               Esta documentación (PDF e imágenes) y sus fuentes
├─ .claude/
│   └─ agents/         Los 17 agentes
│   └─ skills/         Los 4 skills (diseño, ejecución E2E y API, k0lmena)
├─ input/              historias/, documentacion/, api/, bugs/
├─ output/             Lo que generan los agentes
├─ plantillas/         Reporte de bug, planilla de casos, cobertura
├─ scripts/            Casos, reportes HTML, plan, informe de cierre, tablas
│   └─ gestion/        Xray, QMetry (QTM4J) y AIO Tests
├─ herramientas/
│   └─ newman/          Colecciones de Postman
│   └─ k0lmena/
│       └─ web/         features/ steps/ locators/ + framework
│       └─ api/         features/ steps/ (comunes.steps.ts)
│       └─ mobile/      features/ steps/ locators/ support/ apps/
│       └─ performance/ k6/http/ y artillery/
│       └─ reports/     Reportes HTML
│       └─ run-tests.js  Runner de npm test
│       └─ run-perf.js   Runner de npm run perf
```

14 Problemas frecuentes

PROBLEMA	SOLUCIÓN
Un agente nuevo no aparece	Reiniciá Claude Code: los agentes se cargan al iniciar la sesión.
La suite web falla con viewport 0x0 o sin URL	Falta el <code>.env</code> de la raíz o sus variables de k0lmena: <code>cp .env.example .env</code> y completalo.
<code>npm test</code> corre 0 escenarios	Todavía no automatizaste nada (las carpetas vienen vacías) o el filtro <code>TAGS</code> no coincide.
Un escenario nunca se ejecuta	Tiene <code>@bloqueado</code> ; el motivo está en un comentario arriba del escenario.
No encuentro k6	<code>npm run bootstrap:k6</code> en <code>herramientas/k0lmena/</code> o k6 en el PATH.
<code>npm run perf</code> no corre un perfil con carga desde el agente	Es a propósito: fuera de una terminal interactiva hace falta <code>--confirmar</code> , que el agente agrega solo después de tu confirmación.
Una carpeta de Xray/QMetry/AIO aparece como <code>C:/Program Files/Git/...</code>	En Git Bash escribí las carpetas sin <code>/</code> inicial.
Un conector MCP no conecta	Las variables tienen que estar en el entorno donde lanzás <code>claude</code> ; para Atlassian y Figma, autorizá desde <code>/mcp</code> .
Falta una librería de Python	<code>pip install -r requirements.txt</code>

15 Cómo extender el proyecto

Agregar un agente

1. Creá `.claude/agents/mi-agente.md` con frontmatter `name` y `description`. La descripción es lo que usa Claude Code para decidir cuándo invocarlo.
2. Escribí el cuerpo en español: rol, entradas, proceso, salida y reglas.
3. Si genera un formato propio, sumá su plantilla en `plantillas/` o su script en `scripts/`, y su carpeta en `output/`.
4. Si genera tablas en un `.md`, normalizalas con `formatear_tablas.py`.

Agregar un skill, un conector o una herramienta

- **Skill**: una carpeta en `.claude/skills/` con su `SKILL.md`; se carga solo cuando hace falta.
- **Conector MCP**: una entrada nueva en `.mcp.json`; los tokens siempre como `${VARIABLE}`.
- **Herramienta**: una subcarpeta en `herramientas/` con su README.

El detalle está en `ARQUITECTURA.md`.

Regenerar esta documentación

```
node docs/fuente/generar.js # requiere k0lmena instalado (usa su Playwright)
```

Genera `docs/k0lmenIA-arquitectura.png`, `docs/k0lmenIA-uso.png` y `docs/k0lmenIA-documentacion.pdf` a partir de los HTML de `docs/fuente/`.

Licencia y créditos

Licencia **MIT** — © 2026 **QARMY**. k0lmena es un proyecto open source de **Danilo Vezzoni**, con **Gianella Vezzoni**, **Maximiliano Pintos** y **Yanko Leta**. Hecho por la comunidad de **Underc0de**.